

CV10: Dice Game - Zadanie

1 Zadanie cvičenia

Vytvorte Android aplikáciu v jazyku Kotlin s použitím Jetpack Compose, Navigation Compose, `ViewModel` triedy a Room databázy. Aplikácia bude predstavovať jednoduchý *Dice Game*, teda miniaplikáciu na prácu s vlastnými kockami a históriou hodov.

Balík aplikácie použite v tvare `com.example.cv10`.

2 Požadované obrazovky

Aplikácia musí obsahovať tri obrazovky.

Prvá obrazovka bude hlavná obrazovka výberu kocky. Musí zobrazovať predvolenú kocku D6 a zároveň umožniť používateľovi pridať vlastné kocky podľa počtu strán. Používateľ tak bude môcť pridať napríklad D4, D20 alebo D100. Na tejto obrazovke sa musí nachádzať vstupné pole pre počet strán, tlačidlo na pridanie novej kocky a zoznam všetkých dostupných kociek.

Pri každej kocke sa musia nachádzať dve akcie. Prvá akcia otvorí obrazovku hodu pre konkrétnu kocku. Druhá akcia otvorí obrazovku štatistiky pre konkrétnu kocku.

Druhá obrazovka bude obrazovka hodu. Musí zobrazovať konkrétnu vybranú kocku, tlačidlo na vykonanie hodu a výsledok posledného hodu. Po stlačení tlačidla sa má vygenerovať náhodné číslo od 1 po počet strán danej kocky. Výsledok sa musí uložiť do databázy.

Tretia obrazovka bude obrazovka štatistiky. Táto obrazovka musí zobrazovať údaje pre jednu vybranú kocku. Súčasťou štatistiky má byť aspoň počet hodov, posledný výsledok, najvyšší výsledok, najnižší výsledok a priemerná hodnota. Pod týmito údajmi môže byť zobrazený aj zoznam posledných hodov.

3 Požadované správanie aplikácie

Po prvom spustení aplikácie sa má do databázy automaticky pridať kocka D6. Používateľ teda nemá začínať s prázdny zoznamom.

Pri pridávaní novej kocky je potrebné kontrolovať vstup. Počet strán musí byť celé číslo a zároveň musí byť aspoň 2. Ak je vstup neplatný, aplikácia má zobraziť jednoduchú chybovú správu. Zároveň sa nesmie umožniť pridanie duplicitnej kocky s rovnakým počtom strán.

Pri hode kockou sa má výsledok okamžite zobrazíť na obrazovke a zároveň uložiť do databázy. Po návrate na hlavnú obrazovku aj po opätovnom otvorení aplikácie musia byť kocky aj história hodov stále dostupné.

4 Požiadavky na rozloženie kódu

Používateľské rozhranie vytvorte v Jetpack Compose. Navigáciu medzi obrazovkami riešte cez Navigation Compose. Stav a logiku aplikácie spracujte vo `ViewModel` triede. Trvalé uloženie údajov realizujte cez Room databázu.

Aplikácia musí obsahovať dve entity. Prvá entita bude reprezentovať kocku. Druhá entita bude reprezentovať konkrétny záznam o hode. Medzi týmito entitami má existovať jednoduchá väzba cez identifikátor kocky.

Odporúčanou súčasťou riešenia je aj jednoduchý repozitár, ktorý bude sprostredkúvať komunikáciu medzi `ViewModel` triedou a DAO vrstvou.

Vhodná štruktúra súborov môže vyzeráť napríklad takto:

```
MainActivity.kt
DiceEntity.kt
RollEntryEntity.kt
DiceGameDao.kt
DiceGameDatabase.kt
DiceGameRepository.kt
DiceGameUiState.kt
DiceGameViewModel.kt
DiceSelectionScreen.kt
DiceRollScreen.kt
DiceStatisticsScreen.kt
```

5 Odporúčaný dátový model

Entita kocky môže vyzeráť napríklad takto:

```
@Entity(tableName = "dice")
data class DiceEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val sides: Int,
    val createdAt: Long
)
```

Entita hodu môže vyzeráť napríklad takto:

```
@Entity(
    tableName = "roll_entries",
    foreignKeys = [
        ForeignKey(
            entity = DiceEntity::class,
            parentColumns = ["id"],
            childColumns = ["diceId"],
            onDelete = ForeignKey.CASCADE
        )
    ],
    indices = [Index(value = ["diceId"])]
)
data class RollEntryEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val diceId: Long,
    val result: Int,
    val rolledAt: Long
)
```

Na uchovanie časových údajov použite typ `Long`. Vďaka tomu budete môcť jednoducho zapisovať čas a neskôr ho zobrazovať vo formátovanej podobe.

6 Ukážka DAO rozhrania

Minimálne DAO rozhranie môže obsahovať tieto operácie:

```
@Dao
interface DiceGameDao {

    @Query("SELECT * FROM dice ORDER BY sides ASC")
    fun getAllDice(): Flow<List<DiceEntity>>

    @Query(...)
    fun getDiceById(diceId: Long): Flow<DiceEntity?>

    @Query(...)
    fun getRollsForDice(diceId: Long): Flow<List<RollEntryEntity>>

    @Insert
    suspend fun insertDice(dice: DiceEntity): Long

    @Insert
    suspend fun insertRoll(roll: RollEntryEntity)
}
```

7 Ukážka logiky vo ViewModel triede

Vo `ViewModel` triede spracujte pridanie kocky, validáciu vstupu, vykonanie hodu a získanie štatistík. Jedna z dôležitých funkcií bude napríklad samotný hod kockou.

```
fun rollDice(diceId: Long) {  
    viewModelScope.launch {  
        val dice = repository.getDiceByIdSnapshot(diceId) ?:  
            return@launch  
        val result = Random.nextInt(1, dice.sides + 1)  
        repository.insertRoll(diceId, result)  
    }  
}
```

Podobným spôsobom spracujte aj pridanie novej kocky a prípravu dát pre štatistiku.

8 Lifecycle-aware zber dát

V tomto cvičení použijete pri zbere dát z `Flow` aj lifecycle-aware prístup. To znamená, že v `Compose` obrazovkách budete používať `collectAsStateWithLifecycle()` namiesto obyčajného `collectAsState()`.

```
val diceFlow = remember(diceId) { diceGameViewModel.getDiceStream  
    (diceId) }  
val dice by diceFlow.collectAsStateWithLifecycle(initialValue =  
    null)
```

Tento prístup je vhodný preto, že obrazovka zbiera dáta iba vtedy, keď je v stave vhodnom na pozorovanie.

9 Inštalovanie KSP a navigácii

Pri použití Room v Kotlin projekte je potrebné doplniť KSP konfiguráciu. Zároveň je potrebné pridať závislosť pre Navigation Compose a pre lifecycle-aware napojenie Compose obrazoviek na dáta.

Jedna z možných konfigurácií môže vyzeráť takto:

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)
}

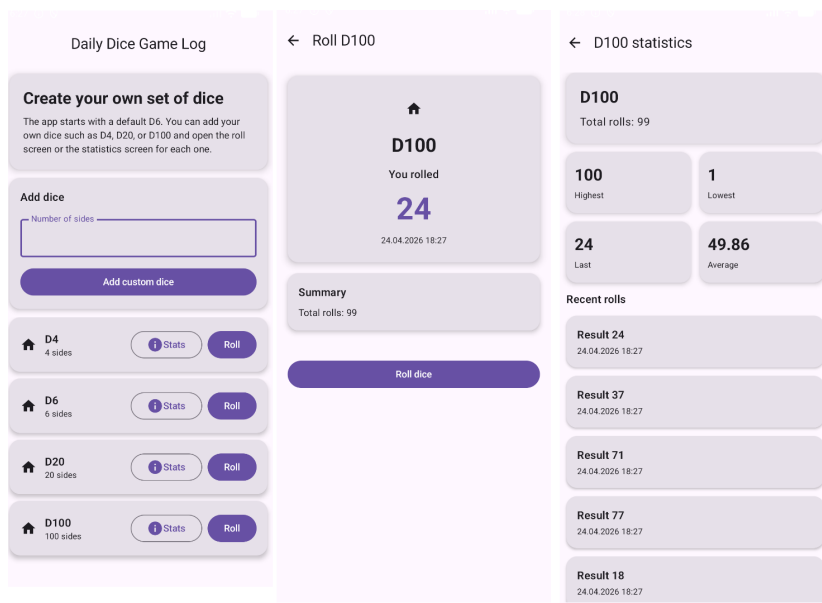
dependencies {
    implementation("androidx.room:room-runtime:2.8.4")
    implementation("androidx.room:room-ktx:2.8.4")
    ksp("androidx.room:room-compiler:2.8.4")

    implementation("androidx.navigation:navigation-compose:2.9.0")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.6")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.6")
}
```

10 Výsledok cvičenia

Po dokončení budete mať viacobrazovkovú miniaplikáciu, v ktorej sa spoja Compose UI, navigácia, ViewModel, Room databáza a lifecycle-aware zber dát. Aplikácia bude obsahovať vlastné kocky, históriu hodov aj odvodené štatistiky pre jednotlivé položky.

Týmto cvičením si upevníte nielen prácu s jednotlivými Jetpack komponentmi, ale aj ich vzájomné prepojenie v rámci jednej menšej aplikácie.



Obr. 1: Ukážka finálnej kockovej hry